

Trabalho Prático 1

TAD Ordenador Universal

Júlia Almeida Luna (202401424)

julialuna@ufmg.br

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - Minas Gerais

2025

1. Introdução

Este trabalho prático tem como objetivo desenvolver uma solução para o produto inovador da empresa Zambs: o Tipo Abstrato de Dados Ordenador Universal, capaz de selecionar de forma automática o algoritmo de ordenação mais eficiente, entre QuickSort e InsertionSort, com base em parâmetros aprendidos e nas características do vetor a ser ordenado.

Para isso, foi implementado um sistema modular que analisa o vetor de entrada e decide qual algoritmo utilizar, considerando o número de quebras (quando um item do vetor é maior que seu sucessor) e o tamanho do vetor. A decisão é fundamentada em uma função de custo que pondera três métricas essenciais: o número de comparações realizadas, o número de movimentações de elementos no vetor e o número de chamadas de funções.

A partir dessa função de custo, determina-se o ponto ótimo, em termos de tamanho e grau de desordem do vetor, a partir do qual o uso do InsertionSort se torna computacionalmente mais vantajoso do que o QuickSort.

2. Implementação

O sistema desenvolvido foi implementado na linguagem C++, utilizando a versão do compilador G++ com padrão C++11

2.1. Classes e Módulos

O código foi dividido em 3 módulos principais, sendo eles:

- **Sorter**: módulo responsável pela implementação dos algoritmos de ordenação propriamente ditos. Contém os métodos de InsertionSort e QuickSort com mediana de três e uso do InsertionSort para pequenas partições.
- **OperationsCounter**: encapsula a contagem de operações de movimentações, comparações e chamadas de funções dos algoritmos de ordenação. Responsável pela contagem de operações fundamentais para o cálculo do custo das ordenações utilizado no processo de calibração dos limiares.
- **UniversalSorter**: módulo que executa os múltiplos testes cujo objetivo é determinar os limiares de número de quebras e tamanho mínimo de partição e coordena a seleção do algoritmo de ordenação mais adequado com base nos parâmetros experimentais definidos nos testes.

2.2. Determinação dos limiares

Um dos maiores desafios do projeto do TAD Ordenador Universal é a determinação do limiar de quebras e do menor tamanho de partição a partir dos quais é mais computacionalmente vantajoso utilizar o algoritmo do InsertionSort ao invés do QuickSort. Esses dois parâmetros são essenciais para garantir o comportamento do Ordenador Universal de ter a capacidade de escolher de maneira automática o algoritmo mais eficiente.

Para realizar a tarefa de escolher qual algoritmo usar em cada situação, o programa usa três funções principais: *determine_break_threshold*, *determine_partition_threshold* e *find_new_range*.

A função *determine_partition_threshold* serve para descobrir qual é o tamanho ideal da “partição” (uma parte do vetor) a partir do qual o programa deve parar de usar o algoritmo QuickSort e passar a usar o insertion sort, que é melhor para partes pequenas.

Para isso, o programa começa testando alguns tamanhos de partição, variando entre o menor possível (2) e o tamanho total do vetor. Entretanto, ele não testa todos os valores da faixa: ele escolhe 6 pontos espaçados uniformemente entre esses limites, ou seja, divide essa faixa em 5, calculando assim o valor do “passo” entre esses valores, e seleciona o novo valor a ser testado somando um passo ao valor testado anteriormente.

Depois da primeira rodada de testes, ele identifica qual tamanho teve o melhor resultado (menor custo de ordenação). Em seguida, ele define uma nova faixa de valores para testar, focando perto desse melhor tamanho utilizando a função *find_new_range*.

A função *find_new_range* escolhe a nova faixa de acordo com o seguinte raciocínio: se o melhor valor estiver perto do começo ou do fim da faixa, o programa ajusta a nova faixa para não ultrapassar os limites válidos (escolhe a faixa que corresponde aos dois primeiros ou os dois últimos passos). Se estiver no meio, ele testa os valores ao redor desse ponto para encontrar um limite ainda melhor (um passo a menos e um passo a mais do ponto). A nova faixa é definida seguindo essas condições e tendo assim o tamanho de dois passos, ou seja, seu tamanho é 40% do tamanho da faixa anterior.

Isso permite uma busca iterativa e controlada para refinamento dos limiares usados na ordenação, mantendo a faixa dentro dos limites válidos e cobrindo a vizinhança do ponto atual.

A figura 1 ilustra o comportamento das funções, em que na primeira iteração variamos os testes entre 2 e 1000 (que é o tamanho do vetor) escolhendo valores espaçados por um passo, que nesse caso é 199.

```
iter 0
mps 2 cost 118.433083300 cmp 11772 move 7904 calls 1489
mps 201 cost 214.459646500 cmp 36789 move 36560 calls 25
mps 400 cost 426.131155000 cmp 74149 move 74537 calls 10
mps 599 cost 724.425434800 cmp 126188 move 126934 calls 4
mps 798 cost 724.425434800 cmp 126188 move 126934 calls 4
mps 997 cost 724.425434800 cmp 126188 move 126934 calls 4
nummps 6 limParticao 2 mpsdiff 307.698059

iter 1
mps 2 cost 118.433083300 cmp 11772 move 7904 calls 1489
mps 81 cost 114.661770100 cmp 18275 move 17051 calls 73
mps 160 cost 200.293099600 cmp 34209 move 33872 calls 28
mps 239 cost 219.875769400 cmp 37923 move 37864 calls 22
mps 318 cost 365.551488100 cmp 63459 move 63669 calls 13
mps 397 cost 426.131155000 cmp 74149 move 74537 calls 10
nummps 6 limParticao 81 mpsdiff 81.860016
```

Figura 1: Exemplo de execução de escolha de partição

Esse processo de ajuste e teste de faixas é repetido até que o programa tenha poucos valores para testar (menos que 5) ou a diferença entre o melhor e o pior custo seja pequena o bastante, conforme o limite definido nas entradas.

Desta forma, o programa vai refinando a escolha do tamanho ideal para alternar entre os algoritmos de ordenação, de forma eficiente e controlada.

A função *determine_break_threshold* também realiza uma série de testes para encontrar o melhor número de quebras, ou seja, o nível de desordem do vetor que serve de referência para decidir qual algoritmo usar.

Nesses testes, a função compara o custo da ordenação do vetor usando QuickSort e InsertionSort para cada valor de quebra testado. O objetivo é identificar o ponto em que os custos dos dois algoritmos ficam mais próximos.

O critério de parada para esses testes e para a escolha das novas faixas de valores a serem testados é o mesmo usado na função que determina o limiar de partição.

A principal diferença entre as duas funções está na faixa inicial de valores testados e no critério para definir o melhor valor. Para o limiar de quebras, a faixa inicial começa em 1 (valor mínimo) e vai até a metade do tamanho do vetor (valor máximo).

Durante os testes, diferente da função de partição, a escolha do melhor número de quebras é feita com base na menor diferença entre os custos dos dois algoritmos. Ou seja, o valor ideal de quebras é aquele para o qual o custo de ordenar o vetor com QuickSort e com InsertionSort são mais próximos, indicando um ponto de equilíbrio para a escolha do algoritmo.

Por fim, a função que realiza de fato a ordenação mais eficiente, a função *universal_sorter*, consiste na lógica de: Caso o número de quebras do vetor seja menor que o do limiar, o InsertionSort é aplicado. Em caso contrário verificamos se o tamanho do vetor é menor que o tamanho mínimo de partição, se isto ocorrer, a função InsertionSort é utilizada para a ordenação, caso o vetor seja maior, utilizamos o QuickSort com InsertionSort para partição menores que o tamanho mínimo de partição, garantido uma ordenação eficiente.

2.3. Estruturas de dados

Durante toda a implementação do Ordenador Universal, foram utilizados vetores dinâmicos de inteiros como estrutura principal para armazenar os dados. Essa escolha se deu pela simplicidade e eficiência que os vetores oferecem para manipulação sequencial, o que facilita a aplicação direta dos algoritmos clássicos de ordenação, como insertion sort e QuickSort. O vetor dinâmico é especialmente útil na fase de leitura dos dados, quando o tamanho do vetor a ser ordenado é determinado por um parâmetro fornecido no arquivo de entrada. Como esse tamanho pode variar a cada execução, a alocação dinâmica permite adaptar o espaço de memória conforme a necessidade, evitando desperdício ou limitações.

Além disso, os vetores dinâmicos também foram utilizados para armazenar as estatísticas geradas durante as execuções das funções *determine_partition_threshold* e *determine_break_threshold*. Nessas funções, o número de testes realizados em cada iteração não é fixo e pode variar dependendo do comportamento dos dados. Com o uso do vetor dinâmico, é garantido que todas as informações importantes serão armazenadas de forma

eficiente, sem desperdício de memória, pois o tamanho do vetor pode crescer conforme necessário.

2.4. Execução do programa

Para executar o programa, é necessário um sistema Linux com o compilador g++ (C++11). No terminal, deve-se navegar até o diretório do projeto e usar o comando `make all` para gerar o executável. A execução requer um arquivo de entrada com os seguintes parâmetros, em ordem: seed, limiar de custo, coeficientes de comparação, chamadas, movimentações, tamanho do vetor e seus elementos. Com isso, o programa é iniciado via:

`bin/tp1.out nome_do_arquivo_de_entrada.txt`

3. Análise de Complexidade

3.1. Complexidade da determinação do tamanho de partição

3.1.1. Complexidade de tempo

A determinação do tamanho mínimo de partição passa por diversas iterações, fundamentais para a análise da complexidade da função *determine_partition_threshold* como um todo. Primeiramente, analisamos o primeiro laço *for* que realiza os testes de partição. Ele percorre os valores desde o menor tamanho de partição da faixa testada, somando o passo até ultrapassar o maior tamanho da faixa. O número de iterações desse laço depende da diferença entre os valores mínimo e máximo da faixa. No pior caso, quando essa diferença é igual a 9, o passo será igual a 1 (já que $(\text{maxMPS} - \text{minMPS}) / 5 \leq 1$), resultando em até 9 iterações, o valor máximo do número de partições (numMPS).

Cada iteração consiste na chamada do Ordenador Universal, com limiar de quebras igual a 0, usamos assim o QuickSort com inserção para partições pequenas. Como esse algoritmo pode, no pior caso, ter custo $O(n^2)$, e as demais operações (registro e impressão de estatísticas) possuem custo constante, podemos concluir que o custo de uma rodada de testes no pior caso é:

$$O(\text{numMPS} * n^2) = O(9 * n^2) = O(n^2)$$

Em segundo lugar, devemos considerar quantas vezes esse conjunto de testes é repetido. Isso é determinado por um laço *while*, cuja condição de parada ocorre quando a diferença de custo entre os extremos da faixa testada se torna menor que um limiar ou quando o número de partições a serem testadas cai abaixo de 5.

É importante observar que, a cada rodada de testes, a faixa de partições a ser avaliada é reduzida. Mais precisamente, apenas $\frac{2}{5}$ da faixa é mantida para a próxima rodada, o que significa que a faixa é reduzida a $\frac{2}{5}$ de seu tamanho anterior. Esse comportamento define a seguinte relação de recorrência:

$$T(n) = T(2n/5) + f(n)$$

Como $f(n) \in O(1)$ (já que não há combinação complexa dos resultados de chamadas anteriores), podemos aplicar o Caso 2 do Teorema Mestre, o que nos leva à conclusão de que esse laço possui complexidade $O(\log(n))$.

Portanto, como temos $O(\log(n))$ rodadas de testes, cada uma com custo $O(n^2)$ no pior caso, a complexidade de tempo total da função *determine_partition_threshold* é:

$$O(n^2 \log(n))$$

3.1.2. Complexidade de espaço

Tendo compreendido a quantidade de tempo que o algoritmo consome no pior caso, podemos agora avaliar quanto espaço de memória ele requer durante sua execução da determinação do limiar de quebras. Isto será determinado por estruturas auxiliares utilizadas durante o teste.

A principal delas é um vetor auxiliar com o mesmo tamanho do vetor de entrada, utilizado para restaurar o estado original da entrada após cada ordenação. Isso é necessário para garantir a consistência dos testes, pois cada ordenação modifica a estrutura do vetor. Assim, este vetor contribui com uma complexidade de espaço de $O(n)$.

Utilizamos também um vetor alocado dinamicamente para guardar as estatísticas de cada valor testado na rodada de testes, como o tamanho máximo de testes é 9 (como visto anteriormente) e o vetor é desalocado ao final de cada iteração, não acumulando memória entre as rodadas. Assim, sua contribuição é constante, ou seja, $O(1)$ por iteração.

Além disso, o espaço da pilha de chamadas do QuickSort precisa ser considerado. No pior caso clássico, o QuickSort pode atingir uma profundidade de $O(n)$.

Temos como resultado da complexidade de espaço total:

$$O(n+1+n) = O(n)$$

3.2. Complexidade da determinação do limiar de quebras

3.2.1. Complexidade de tempo

A lógica da função *determine_break_threshold* é bastante semelhante à da função *determine_partition_threshold*, especialmente no uso da estrutura de repetição principal.

Ambas utilizam um laço *while* cuja condição de parada ocorre quando a diferença de custo entre os extremos da faixa testada se torna menor que um limiar pré-definido ou quando o número de valores testados cai abaixo de 5. Como visto anteriormente, a complexidade desse laço é $O(\log n)$.

Além disso, ambas funções utilizam o mesmo laço *for*, responsável por iterar sobre os valores da faixa corrente, que como discutido anteriormente, possui custo constante e sua complexidade é $O(1)$.

A principal diferença entre as duas funções está no conteúdo da rodada de testes. Na *determine_break_threshold*, para cada valor de quebra testado:

- O vetor é embaralhado com base no número de quebras desejado,. A função *shuffleVector* percorre o vetor e executa $O(i)$ trocas (cujo custo é $O(1)$), onde i é o número de quebras, o que representa custo $O(i)$.
- O vetor embaralhado é então ordenado duas vezes: uma com QuickSort e outra com InsertionSort, ambas no pior caso com custo $O(n^2)$.
- As funções de registro e impressão de estatísticas são chamadas a cada teste e possuem custo constante, $O(1)$.

Portanto, o custo de uma rodada de testes é:

$$O(n^2 + i + 1) = O(\max(n^2+i+1)) = O(n^2)$$

Determinamos que assim como na função de determinação do limiar de quebras o custo é:

$$O(n^2 \log(n)).$$

3.2.2. Complexidade de espaço

Para a execução da função, utilizamos, assim como na determinação do tamanho de partição, dois vetores para guardar as estatísticas da ordenação com QuickSort e com insertion sort. Como máximo de testes é 9 e os vetores são desalocados ao final de cada iteração, seu custo é constante, ou seja, $O(1)$ por iteração.

Diferentemente da função *determine_partition_threshold*, para determinar o limiar de quebras não utilizamos um vetor de cópia, devido ao fato de não ser necessário restaurar a configuração do vetor antes do ordenamento, uma vez que para embaralhar o vetor é necessário que ele esteja ordenado. Assim, apenas aproveitamos o vetor já ordenado pelos testes anteriores e fazemos o embaralhamento antes de cada teste, evitando o uso de mais memória auxiliar.

Consideraremos também a complexidade do pior caso do QuickSort chamado na rodada de testes que é $O(n)$. Temos como complexidade total então:

$$O(1+n) = O(n)$$

3.3. Complexidade do Ordenador Universal

A complexidade da função *universal_sorter* depende diretamente dos valores de limiar de quebras e tamanho mínimo de partição fornecidos como parâmetros. Se esses valores não forem ideais (como ocorre durante os testes da função *determine_partition_threshold*, onde o limiar de quebras é fixado em 0), o algoritmo pode acabar executando o pior caso do QuickSort ou do InsertionSort, a depender da situação do vetor.

Nessas condições, a função pode atingir a complexidade de tempo $O(n^2)$, que corresponde ao pior caso de ambos os algoritmos. Quanto ao uso de memória, o pior caso ocorre no QuickSort devido ao uso de chamadas recursivas e estrutura auxiliar, resultando em uma complexidade de espaço $O(n)$.

4. Estratégias de Robustez

Para garantir a robustez e a segurança da execução do TAD, foram implementadas diversas estratégias que evitam comportamentos inesperados durante a execução. Dentre elas, destacam-se as verificações de erro no acesso a arquivos, com mensagens claras quando o nome do arquivo de entrada não é fornecido ou quando há erros na abertura do arquivo. Destaca-se também a validação de índices antes do acesso a vetores para evitar violações de memória, onde apenas acessamos o índice caso ele seja maior ou igual a 0 e menor que o tamanho do vetor. Além disso, o código também verifica se a alocação dinâmica de memória foi bem-sucedida antes de utilizar ponteiros, garantindo que recursos não acessíveis não sejam utilizados e, caso contrário, uma mensagem de erro é exibida e a execução é interrompida de forma segura.

5. Análise Experimental

Com o objetivo de verificar a eficiência do Ordenador Universal, foram realizados diversos testes com o TAD desenvolvido, variando-se três dimensões que influenciam diretamente o tempo de execução: o tamanho do vetor, o tamanho da chave de comparação e o tamanho do registro.

Os mesmos testes foram aplicados aos algoritmos InsertionSort e QuickSort em suas versões não otimizadas, permitindo a comparação direta e a avaliação de eventuais vantagens computacionais do Ordenador Universal.

Cada experimento foi realizado variando uma das três dimensões citadas, em quatro cenários distintos de desordem no vetor: um vetor totalmente ordenado, um vetor com 40% de quebras, um com 60% e outro com 100% de quebras. A estratégia de geração de quebras seguiu a mesma lógica utilizada nos testes de determinação do limiar de quebras.

Essas variações foram aplicadas sobre um vetor base: um vetor ordenado de tamanho 1000, com chave do tipo char de tamanho 10 e registros do tipo char de tamanho 40. Isto foi feito para padronizar a análise e verificar as influências das dimensões citadas:.

Para cada configuração testada, os algoritmos QuickSort e InsertionSort foram executados diversas vezes, e os tempos de execução foram registrados, juntamente com o número de comparações, chamadas de função e movimentações realizadas. A partir desses dados, aplicou-se uma regressão linear pelo método dos Mínimos Quadrados com fatoração QR, obtendo-se os coeficientes utilizados na função de custo do Ordenador Universal.

Com base nestes coeficientes, foi possível determinar o tamanho mínimo de partição e o limiar de quebras ideais (permitindo que o Ordenador Universal selecione automaticamente o algoritmo de ordenação mais eficiente para cada cenário) e calcular o custo de cada algoritmo nos testes.

5.1. Teste da variação de chave

O primeiro cenário avaliado consistiu na variação do tamanho da chave, que passou de 10 para 40 e, posteriormente, para 90 bytes. O tamanho do registro e do vetor mantiveram-se como o vetor base.

Como era esperado, o custo da ordenação aumentou proporcionalmente ao crescimento do tamanho da chave, já que comparações entre elementos tornam-se mais custosas à medida que a quantidade de dados a ser comparada cresce.

Os resultados obtidos nesses experimentos estão organizados na Tabela 1.

Desordem	Chave 10	Chave 40	Chave 90
0% Quebras			
Universal Sorter	10.956079	23.449711	54.191784
Quick Sort	181.271881	302.754974	412.965576
Insertion Sort	10.956079	23.449711	54.191784
40% Quebras			
Universal Sorter	73.766319	89.076706	129.187439
Quick Sort	257.449188	354.103577	707.469788
Insertion Sort	1002.495605	2506.811523	2801.487061
60% Quebras			
Universal Sorter	82.533867	116.752220	161.348267
Quick Sort	295.337799	412.249695	818.123535
Insertion Sort	1177.158936	2940.392334	3285.433350

100% Quebras			
Universal Sorter	89.480865	142.801483	193.510788
Quick Sort	310.256531	451.006439	878.523926
Insertion Sort	1426.857544	3560.239502	3977.282715

Tabela 1: custos dos experimentos de variação do tamanho de chave

5.2. Teste da variação de registro

O segundo experimento analisou a influência do tamanho do registro no custo da ordenação. Para isso, variou-se o tamanho dos registros de 40 para 200 e, em seguida, para 500 bytes, mantendo-se as demais características do vetor base inalteradas.

Os resultados, apresentados na Tabela 2, mostram um aumento no custo da ordenação à medida que o tamanho do registro cresce. Esse comportamento é justificado pelo maior custo associado à movimentação de registros maiores durante a ordenação.

Desordem	Registro 40	Registro 200	Registro 500
0% Quebras			
Universal Sorter	10.956079	12.398390	113.668167
Quick Sort	181.271881	288.952606	142.995834
Insertion Sort	10.956079	12.398390	113.668167
40% Quebras			
Universal Sorter	73.766319	195.987808	319.593018
Quick Sort	257.449188	352.075470	390.167480
Insertion Sort	1002.495605	2468.208740	9012.791016
60% Quebras			
Universal Sorter	82.533867	210.655060	382.186707
Quick Sort	295.337799	400.913544	458.849243
Insertion Sort	1177.158936	2900.808838	10580.404297
100% Quebras			
Universal Sorter	89.480865	214.881332	446.894165
Quick Sort	310.256531	412.815613	517.122131
Insertion Sort	1426.857544	3519.253662	12821.463867

Tabela 2: custos dos experimentos de variação do tamanho do registro

5.3. Teste da variação do tamanho do vetor

O terceiro experimento aborda a influência do tamanho do vetor como um todo no custo da ordenação. O vetor foi variado do tamanho 1000 para o tamanho 1500 e 2000, as demais características do vetor base foram mantidas.

Conforme apresentado na Tabela 3, os resultados evidenciam um aumento gradual no custo de ordenação proporcional ao crescimento do vetor, o que era esperado dado o impacto direto do número de elementos nas operações realizadas pelo algoritmo.

Desordem	Tamanho 1000	Tamanho 1500	Tamanho 2000
0% Quebras			
Universal Sorter	10.956079	13.624375	37.495003
Quick Sort	181.271881	390.103058	393.609161
Insertion Sort	10.956079	13.624375	37.495003
40% Quebras			
Universal Sorter	73.766319	262.431580	399.756744
Quick Sort	257.449188	417.680084	522.547241
Insertion Sort	1002.495605	4555.200684	10785.605469
60% Quebras			
Universal Sorter	82.533867	265.327362	433.244598
Quick Sort	295.337799	447.594055	555.666260
Insertion Sort	1177.158936	5457.917480	13492.811523
100% Quebras			
Universal Sorter	89.480865	324.392914	460.675781
Quick Sort	310.256531	497.378540	584.722656
Insertion Sort	1426.857544	6482.876953	15124.083984

Tabela 3: custos dos experimentos de variação do tamanho do vetor

5.4. Análise do Ordenador Universal

Os experimentos realizados com as variações nas dimensões do vetor confirmam que o principal objetivo do projeto foi alcançado: desenvolver um ordenador capaz de escolher automaticamente o algoritmo de ordenação mais eficiente, otimizando o processo de ordenação.

Em todos os cenários testados, o Ordenador Universal apresentou custos de ordenação inferiores aos dos algoritmos QuickSort e InsertionSort não otimizados, demonstrando sua capacidade de determinar com precisão o limiar de quebras e o tamanho mínimo de partição a partir dos quais o uso do InsertionSort se torna mais vantajoso. Dessa forma, a ferramenta se mostra eficaz na escolha adaptativa do algoritmo, promovendo uma ordenação mais eficiente em diferentes contextos. A Figura 2 mostra que, com a variação do tamanho da chave, o Universal Sorter apresentou menor custo que os algoritmos não otimizados.

É válido ressaltar que os processos de calibração dos limiares possuem complexidade de tempo $O(n^2 \log(n))$ e de espaço $O(n)$ cada um. Entretanto, em cenários que a ordenação de vetores similares serão feitas diversas vezes o uso dos limiares no Ordenador Universal trará um ganho de desempenho considerável, tornando-se vantajoso esse investimento inicial na identificação dos limiares.

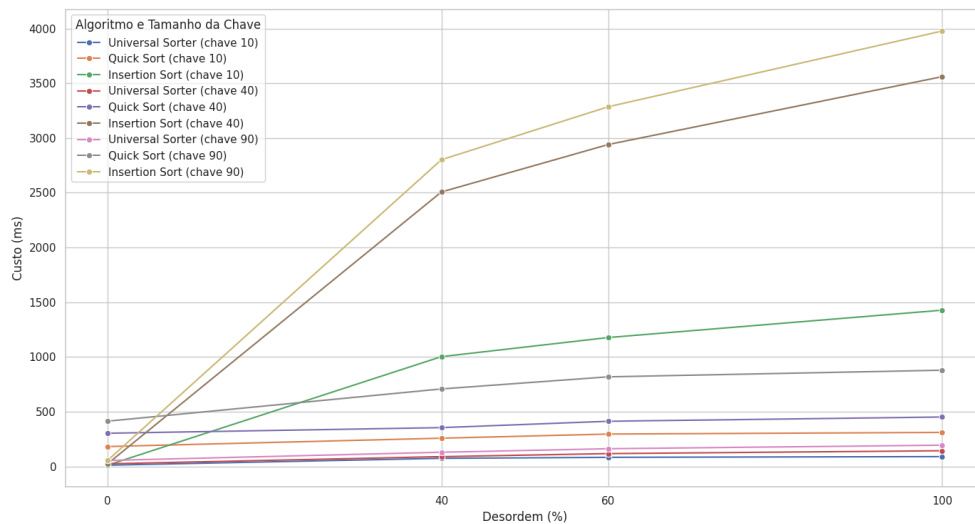


Figura 2: variações de custo pelo aumento da chave

6. Conclusão

Neste trabalho, foi desenvolvido o TAD Ordenador Universal, uma solução capaz de selecionar automaticamente o algoritmo de ordenação mais eficiente entre QuickSort e InsertionSort. Essa escolha é baseada em parâmetros determinados de forma iterativa experimental e nas características do vetor de entrada, o que garante uma ordenação mais eficiente em diferentes cenários.

Ao longo da execução do projeto, diversos conceitos abordados em aula puderam ser observados na prática, como a influência do número de quebras e do tamanho do vetor no desempenho do InsertionSort. Também foi possível aplicar, de forma concreta, a análise de complexidade de tempo e espaço em funções reais, reforçando a importância de buscar soluções com menor custo computacional sempre que possível.

Outro aspecto relevante foi a realização dos experimentos, que permitiu compreender como projetá-los corretamente, interpretar resultados de forma crítica e, principalmente, observar na prática comportamentos que até então eram apenas teóricos. Essa vivência contribuiu significativamente para consolidar o aprendizado e aprofundar o entendimento sobre algoritmos de ordenação e análise de desempenho.

7. Bibliografia

- [1] A. Lacerda e W. Meira Jr., Slides virtuais da disciplina de Estruturas de Dados, 2020. Disponível via Moodle.
- [2] T. H. Cormen, C. E. Leiserson e R. L. Rivest, Algoritmos. 3ª ed., 2017.
- [3] WSCube Tech, "Time and Space Complexity of Sorting Algorithms," [Online]. Disponível em: <https://www.wscubetech.com/resources/dsa/time-space-complexity-sorting-algorithms>. [Acessado: maio 2025].